Refactorizando el caos en SoftMarket

Propuesta integral de buenas prácticas de desarrollo

Informe técnico académico

Datos de entrega	Información
Empresa analizada	SoftMarket S.A.
Autor	Santiago Restrepo Cardona
Modalidad	Entrega individual
Fecha	16 de octubre de 2026

Este informe diagnostica los problemas de ingeniería descritos para SoftMarket y propone un forma de trabajo más ordenada y fácil de mantener para cambios de software. La recomendación central es integrar control de versiones con revisiones obligatorias, pruebas automatizadas y diseño modular. Se desarrolla el ejemplo de gestión de clientes aplicando SOLID y se muestra el recorrido del cambio desde la tarea hasta producción.

Resumen ejecutivo

SoftMarket presenta síntomas de crecimiento sin acuerdos técnicos compartidos: duplicación, regresiones, colaboración frágil y documentación insuficiente. El aumento reportado del 40% en errores de producción durante los últimos tres meses evidencia un problema de calidad que requiere una respuesta de proceso y de diseño. La cifra indica una tendencia informada en el caso; no identifica por sí sola una causa única.

Se propone flujo de trabajo con GitHub con ramas breves por tarea, solicitudes de revisión obligatorios, integración continua y despliegue controlado. Para reducir el dependencia entre partes, el ejemplo de gestión de clientes separa reglas de negocio, guardado de datos y notificaciones mediante interfaces. El plan se implanta en etapas, midiendo resultados con indicadores verificables.

Objetivos de la propuesta

• Reducir regresiones mediante pruebas y revisión sistemática de cambios.

• Facilitar cambios aislados con módulos bien enfocados y dependencias invertidas.

• Hacer reproducible el historial de decisiones y releases con Git.

• Crear documentación breve y operativa que se actualice junto con el código.

1. Diagnóstico del problema

La información disponible describe síntomas, no inventaría repositorios ni identifica tecnologías concretas. Por eso, los casos siguientes son ejemplos plausibles que permiten explicar el riesgo; no se presentan como hechos ya comprobados en SoftMarket.

Área	Problema observado	Ejemplo y efecto
Arquitectura y diseño	Módulos duplicados y dependencias poco claras.	Validaciones de clientes repetidas en ventas y facturación pueden divergir; una corrección parcial mantiene comportamientos inconsistentes.
Versionamiento	No existe flujo Git acordado.	Cambios directos a la rama principal pueden mezclarse, perderse o llegar sin revisión; cuesta saber qué versión contiene cada cambio.
Colaboración	Conflictos y acuerdos implícitos.	Dos personas editan la misma lógica sin coordinación; resolver el conflicto textual no garantiza resolver el conflicto funcional.
SOLID y patrones	Responsabilidades concentradas y abstracciones ausentes.	Un servicio que valida, guarda, envía correos y construye respuestas tiene muchos motivos para cambiar y es difícil probar sin infraestructura real.
Calidad	Pruebas y criterios de aceptación ausentes.	Una modificación en el cálculo o estado de una factura puede romper un flujo existente sin detección antes de producción.
Documentación	Decisiones y operación quedan en conocimiento individual.	Un nuevo integrante desconoce cómo ejecutar, configurar y desplegar el sistema; aumenta el tiempo de incorporación y el riesgo operativo.

Impacto esperado de mantener la situación

La duplicación incrementa el costo de corregir y evolucionar. El dependencia entre partes amplifica el radio de impacto de cada cambio; la falta de pruebas reduce la confianza para refactorizar. Sin historial revisable ni releases trazables, el diagnóstico de incidentes y la reversión se vuelven lentos. Estos factores pueden afectar mantenibilidad, escalabilidad del equipo y disponibilidad, aunque el caso no cuantifica por separado cada impacto.

2. Propuesta de versionamiento con GitHub

Se recomienda flujo de trabajo con GitHub porque mantiene una rama principal protegida y usa ramas breves por cambio. Es adecuado para una startup que necesita integrar con frecuencia y puede desplegar en ciclos pequeños. Si la organización requiere releases con ventanas fijas o varias versiones soportadas simultáneamente, se pueden añadir ramas release bajo una política explícita.

2.1 Ramas y reglas

Rama o referencia	Uso	Regla
main	Código integrado y potencialmente desplegable.	Protegida: no aceptar push directo; requiere pull request, verificaciones aprobadas y aprobación.
mejora/<id>-<tema>	Funcionalidad nueva.	Crear desde main; mantener corta; eliminar tras integrar.
arreglo/<id>-<tema>	Corrección planificada.	Incluye prueba que reproduce el defecto.
arreglo urgente/<id>-<tema>	Corrección urgente de producción.	Mismo control mínimo de revisión y pruebas; desplegar y luego sincronizar main.
tag vX.Y.Z	Release identificable.	Etiqueta anotada vinculada a registro de cambio y notas de cambios.

2.2 Convenciones de registro de cambios

Usar Conventional Commits: tipo, alcance opcional y resumen imperativo en minúscula. El cuerpo explica el motivo o impacto; BREAKING CHANGE describe incompatibilidades. Cada registro de cambio debe representar una unidad comprensible, sin incluir cambios de formato ajenos.

```
feat(clientes): permitir búsqueda por correo
fix(facturacion): evitar duplicar el cobro al reintentar
test(clientes): cubrir correo repetido
docs(api): documentar respuesta de creación
refactor(clientes): separar guardado de datos del servicio
```

2.3 Revisión, conflictos y protección

• Abrir un pull request con contexto, tarea relacionado, alcance, evidencia de pruebas y riesgos; usar una plantilla común.

• Exigir al menos una aprobación de otra persona y aprobación del responsable del área cuando el cambio afecte una regla crítica. El autor no aprueba su propio cambio.

• Automatizar formato, análisis estático, pruebas unitarias e integración. Bloquear la integración si un check falla o hay conversaciones pendientes.

• Mantener solicitudes de revisión pequeños. Revisar propósito, diseño, legibilidad, seguridad, pruebas y documentación, no solo estilo.

• Antes de integrar, actualizar la rama con main y resolver conflictos entendiendo ambos comportamientos; ejecutar de nuevo las pruebas y pedir nueva revisión si cambió la lógica. No resolver conflictos aceptando todo un lado sin inspección.

• Integrar mediante integración en un solo registro para conservar un registro de cambio claro por pull request; eliminar la rama al terminar. Para cambios coordinados, documentar el orden de integración.

2.4 Flujo completo propuesto

```
Tarea priorizada en GitHub Issues
        |
Crear mejora/<id>-<tema> desde main actualizado
        |
Implementar cambio pequeño + pruebas + documentación necesaria
        |
Commit convencional y push de la rama
        |
Abrir Pull Request con contexto y criterios de aceptación
        |
revisión automática: formato + análisis estático + pruebas + compilación
        |
¿Checks verdes y aprobación? -- No -> corregir / conversar / volver a revisión automática
        | Sí
        |
Squash merge a main y eliminar rama
        |
Despliegue automático a pruebas internas + pruebas rápidas / aceptación
        |
Aprobación de release según riesgo -> tag vX.Y.Z -> producción gradual
        |
Monitorear métricas y errores; si falla, volver a la versión anterior y registrar incidente
```

Este diagrama define controles en cada transición. La aprobación funcional se aplica en pruebas internas; para cambios de bajo riesgo puede automatizarse una vez que el equipo demuestre estabilidad y tenga volver a la versión anterior probado.

2.5 Repositorio simulado

La siguiente estructura ejemplifica cómo organizar el repositorio y cómo se vería el historial. Es una propuesta simulada; no se ha creado un repositorio remoto real.

```
softmarket-app/
+-- README.md
+-- CONTRIBUTING.md
+-- docs/
|   +-- architecture.md
|   +-- runbooks/deploy.md
+-- src/clientes/
|   +-- domain/
|   +-- application/
|   +-- infrastructure/
+-- tests/
+-- .github/
    +-- PULL_REQUEST_TEMPLATE.md
    +-- workflows/ci.yml

main:       A--B------M1--------M2--v1.2.0
                 \    /           \   /
mejora/42:       C--D             E-F
arreglo/57:                             G
```

M1 y M2 representan solicitudes de revisión integrados mediante squash. Las ramas mejora/42 y arreglo/57 nacen desde main, pasan por revisión automática y revisión y se eliminan después del merge; la etiqueta v1.2.0 identifica el release.

3. Refactorización de gestión de clientes aplicando SOLID

Problema de ejemplo: una clase ClienteService valida datos, consulta una base SQL, envía correo y registra auditoría. Una variación de la base, del proveedor de correo o de las reglas de alta obliga a modificar la misma clase y dificulta aislar pruebas.

Diseño propuesto

```
Cliente (entidad)
  id, nombre, correo

RegistrarCliente (proceso principal)
  depende de ClienteRepository y NotificadorCliente
  valida reglas de negocio, guarda y solicita notificación

ClienteRepository (contrato)
  obtenerPorCorreo(correo), guardar(cliente)

NotificadorCliente (contrato)
  clienteRegistrado(cliente)

SqlClienteRepository / EmailNotificador (componentes de conexión)
  implementan los contratos; se configuran desde composición de la aplicación
```

Pseudocódigo

```
interface ClienteRepository {
  obtenerPorCorreo(correo): Cliente?
  guardar(cliente): void
}

interface NotificadorCliente {
  clienteRegistrado(cliente): void
}

class RegistrarCliente {
  constructor(repo: ClienteRepository, notificador: NotificadorCliente)

  ejecutar(datos): Cliente {
    si datos.nombre está vacío o correo inválido: error Validacion
    si repo.obtenerPorCorreo(datos.correo) existe: error CorreoDuplicado
    cliente = Cliente.nuevo(datos.nombre, datos.correo)
    repo.guardar(cliente)
    notificador.clienteRegistrado(cliente)
    retornar cliente
  }
}
```

Principio	Aplicación en el ejemplo	Beneficio
S — Responsabilidad única	El proceso principal coordina el registro; entidad contiene reglas propias; componentes de conexión hacen guardado de datos o notificación.	Cada pieza cambia por una razón coherente y se entiende con menor esfuerzo.
O — Abierto/cerrado	Se añaden componentes de conexión, como notificación SMS, implementando el contrato.	Se extiende comportamiento sin alterar reglas estables del proceso principal.
L — Sustitución de Liskov	Cualquier ClienteRepository debe respetar contrato de búsqueda, guardado y errores documentados.	Las implementaciones sustituibles no cambian las expectativas del proceso principal.
I — Segregación de interfaces	Puertos pequeños con operaciones específicas; el servicio no depende de una interfaz genérica de infraestructura.	Los clientes del contrato no implementan métodos que no usan.
D — Inversión de dependencias	RegistrarCliente depende de abstracciones; la composición conecta componentes de conexión concretos.	Reglas de negocio quedan aisladas de SQL y del proveedor de correo.

Consideraciones de implementación

La guardado de datos y la notificación suelen requerir consistencia transaccional. En una versión inicial, definir qué ocurre si guardar funciona y enviar correo falla: preferir registrar el evento pendiente en una tabla de salida transaccional y procesarlo de forma reintentable (patrón bandeja de eventos pendientes) cuando el volumen o la criticidad lo justifiquen. Evitar introducir el patrón antes de que la necesidad exista. El manejo de errores, validación de correo, privacidad y autorización deben definirse con los requisitos reales.

4. Buenas prácticas complementarias

Práctica	Lineamiento propuesto
Estilo	Acordar formatter y linter por lenguaje; automatizar su ejecución local y en revisión automática; nombres descriptivos, funciones acotadas y formato uniforme.

Práctica	Lineamiento propuesto
Pruebas	Pirámide: unitarias para reglas, integración para guardado de datos/contratos y pocas pruebas de extremo a extremo para flujos críticos. Cubrir caminos de éxito, duplicados, validaciones y fallos externos.
Documentación	README con propósito, requisitos, instalación y ejecución; CONTRIBUTING con ramas, registro de cambios y PR; ADR para decisiones arquitectónicas; descripción de la API para APIs; runbook para despliegues e incidentes.
Calidad y seguridad	revisión automática obligatoria, revisión de dependencias y secretos, análisis estático, escaneo de vulnerabilidades y configuración por entorno. No guardar credenciales en el repositorio.
Operación	Despliegue gradual si aplica, health checks, logs estructurados y métricas de errores; volver a la versión anterior practicado. Registrar incidentes y acciones de prevención sin culpar personas.

Indicadores para comprobar avance

Indicador	Cómo observarlo	Meta inicial sugerida
Tasa de fallos de cambios	Cambios desplegados que causan volver a la versión anterior, arreglo urgente o incidente / cambios desplegados.	Establecer línea base en el primer mes; reducirla en los siguientes dos trimestres.
Tiempo de entrega	Mediana desde primer registro de cambio hasta despliegue.	Medir por tipo de cambio y reducir esperas sin saltar controles.
Tiempo de recuperación	Mediana desde detección hasta recuperación del servicio.	Definir objetivo por criticidad y practicar volver a la versión anterior.
Salud de revisión automática	Porcentaje de ejecuciones principales verdes y causas de falla.	Mantener checks rápidos y confiables; corregir pruebas inestables.

Las metas son orientativas: el equipo debe medir su línea base antes de comprometer porcentajes, ya que el caso no aporta valores actuales salvo el aumento del 40% en errores reportado.

5. Plan de adopción

Etapa	Acciones	Resultado esperado
Semanas 1–2	Elegir piloto; documentar reglas Git; proteger main; añadir plantilla PR y revisión automática mínima; definir línea base de métricas.	Primer flujo reproducible en un repositorio.
Semanas 3–6	Aplicar el patrón al módulo de clientes; crear pruebas para reglas críticas; formar al equipo en revisión y resolución de conflictos.	Menor dependencia entre partes y cambios revisables.
Semanas 7–10	Extender prácticas a facturación y otros módulos; añadir documentación de arquitectura y runbooks; revisar dependencias.	Consistencia entre proyectos prioritarios.
Desde semana 11	Revisar métricas mensualmente; priorizar deuda técnica; ajustar reglas según evidencia e incidentes.	Mejora continua sin burocracia innecesaria.

Asignación operativa para un equipo pequeño

Aunque el ejercicio es individual, el proceso está diseñado para un equipo de desarrollo. En un equipo de tres, una persona puede implementar, otra revisar y una tercera validar criterios de aceptación o actuar como responsable de producto; los roles rotan. En una entrega individual, el autor puede simular ramas, PR y revisión automática, pero la revisión de otra persona requiere un colaborador real cuando el proceso se aplique en producción.

Conclusiones

SoftMarket necesita tratar el aumento de defectos como un problema del sistema de trabajo: acuerdos de diseño, integración frecuente, revisión útil, pruebas y trazabilidad. flujo de trabajo con GitHub hace visibles los cambios y establece puertas de calidad sin añadir ramas duraderas innecesarias. El refactor propuesto concentra las reglas del registro de clientes y desacopla guardado de datos y notificaciones mediante contratos claros.

La adopción debe comenzar con un módulo piloto, medir la línea base y ampliar lo que demuestre valor. Los controles no sustituyen el criterio del equipo: deben mantener el feedback rápido, hacer los fallos visibles y permitir recuperarse con seguridad.

Referencias técnicas

• Martin Fowler. “FeatureBranch” y “Continuous Integration”. https://martinfowler.com/bliki/FeatureBranch.html y https://martinfowler.com/articles/continuousIntegration.html

• GitHub Docs. About solicitudes de revisión, protected ramaes y GitHub flow. https://docs.github.com/

• Conventional Commits 1.0.0. https://www.conventionalregistro de cambios.org/en/v1.0.0/

• Robert C. Martin. Agile Software Development: Principles, Patterns, and Practices. Prentice Hall, 2002. (Principios SOLID).

Anexo: lista de verificación de pull request

• [ ] El cambio responde a una tarea y tiene alcance acotado.

• [ ] Las reglas relevantes tienen pruebas; la prueba falla antes de la corrección cuando se trata de un defecto.

• [ ] revisión automática está verde y no hay conversaciones pendientes.

• [ ] Se revisaron compatibilidad, seguridad, datos sensibles y manejo de errores.

• [ ] README, contrato API o runbook se actualizaron si cambió la forma de uso u operación.

• [ ] Existe plan de despliegue y reversión proporcional al riesgo.